

Reinforcement Learning and Decision-making

Hoda Zamani

PhD in Artificial Intelligence and Robotics

GitHub: [hodazamani.github.io](https://github.com/hodazamani)

[Hoda Zamani - Google Scholar](#)

Primary Textbook

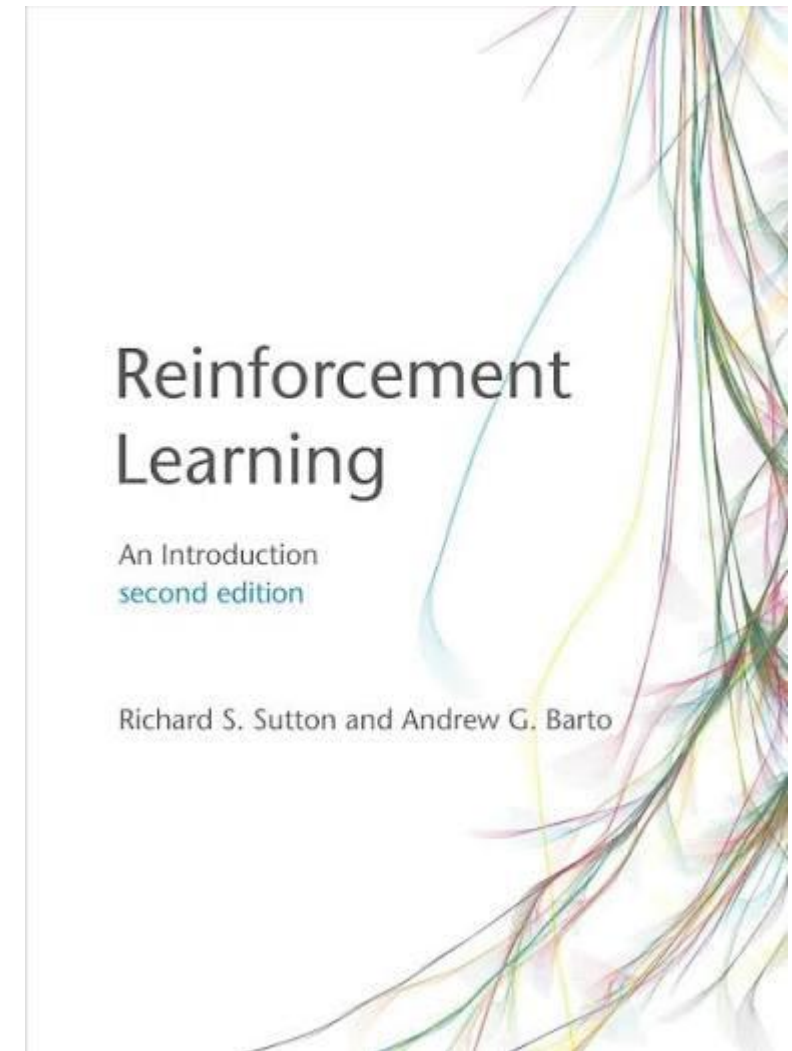
Reinforcement Learning: An Introduction, Second edition, in progress
Richard S. Sutton and Andrew G. Barto, [Download from:
SuttonBartoIPRLBook2ndEd.pdf](#)

Recommended References

- Szepesvári, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool Publishers.
- Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.
- Puterman, M. L. (2014). Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley.

Additional Course Materials

- Instructor-provided lecture notes, slides, and supplementary reading materials.
- Programming assignments and practical exercises using Python and relevant machine learning libraries.
- Selected research papers and technical articles on reinforcement learning and decision-making.
- Computational demonstrations, datasets, and examples provided throughout the course.



You can find:

-  Course announcements & communication
-  Slides & weekly schedule
-  Textbook & reading materials
-  TA
-  Course policies

Here: <https://hodazamani.github.io/Courses/>

Assessment Component	Weight
Weekly Assignments & Programming Exercises	20%
Midterm Examination	30%
Reinforcement Learning Project/Presentation	20%
Final Examination	30%
Total	100%

Mathematics

- Linear algebra
- Probability and statistics
- Basic calculus
- Basic optimization
- Optional: Markov chains, stochastic processes, numerical methods

Machine Learning

- Basic machine learning concepts
- Supervised learning
- Neural networks and deep learning basics
- Familiarity with Python and NumPy

We will soon have a Homework 0 designed to help you review the mathematical and machine learning tools needed for the course.

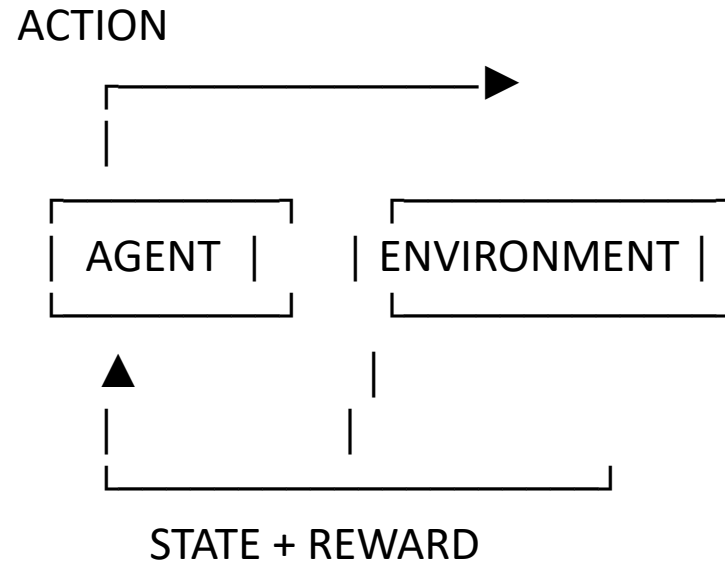
Learning Objectives

By the end of this chapter, you should be able to:

- Define Reinforcement Learning (RL).
- Explain the interaction between an Agent and an Environment.
- Define State, Action, Reward, and Policy.
- Explain Return and the Discount Factor.
- Understand Exploration vs. Exploitation.
- Define a Markov Decision Process (MDP).
- Explain the Markov Property.
- Calculate discounted returns.
- Distinguish RL from supervised learning.

Reinforcement Learning is a framework in which an **agent learns how to make decisions by interacting with an environment and receiving rewards**. The agent does not receive the correct answer directly. Instead, **the agent learns through trial and error**.

Example: Imagine a robot that must find its way to a target.
At first, the robot does not know which path is best. It tries different actions:



The agent:

1. Observes the current state.
2. Choose an action.
3. Receives a reward.
4. Moves to a new state.
5. Learns from this experience.

- The fundamental RL interaction is:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

where:

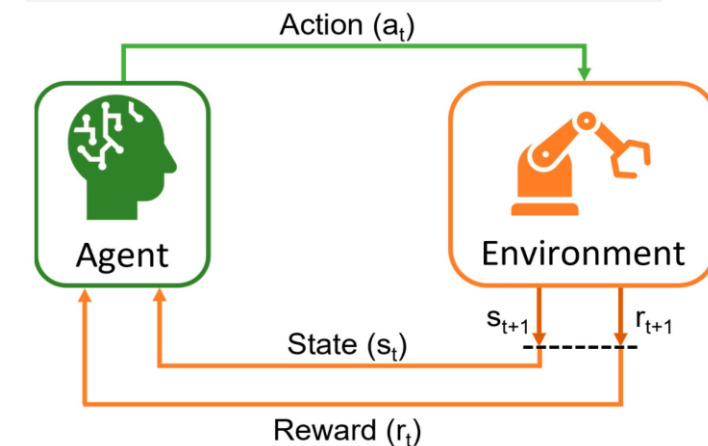
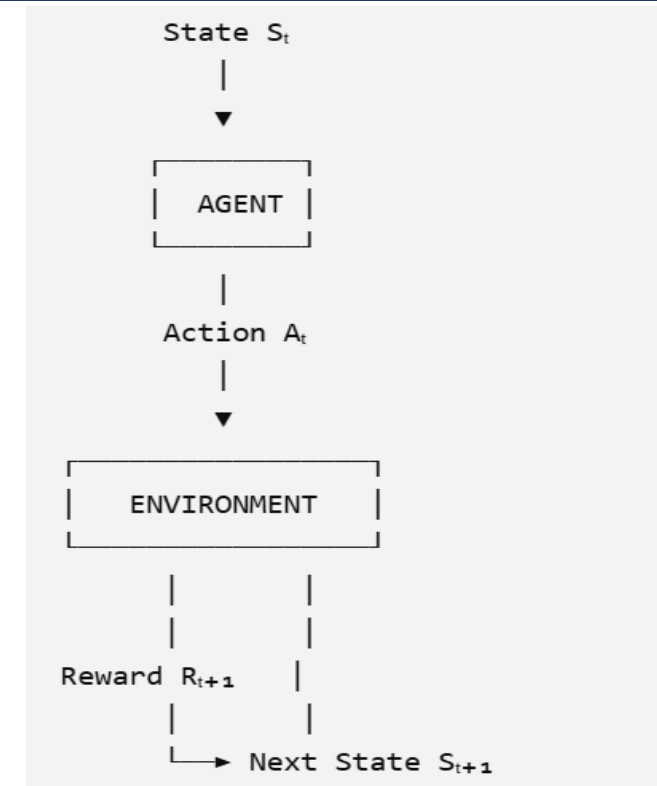
- S_t = current state
- A_t = action selected by the agent
- R_{t+1} = reward received
- S_{t+1} = next state

This loop continues throughout the learning process.

$$S_t \xrightarrow{A_t} S_{t+1}, \quad R_{t+1}$$

or
or

$$S_t \xrightarrow{A_t} (R_{t+1}, S_{t+1})$$



The Agent

The Agent is the decision-maker.

The agent chooses an action according to its policy, which can be expressed as:

$$A_t \sim \pi(a|S_t)$$

Examples:

Problem

Chess

Robot navigation

Video game

Autonomous driving

Temperature control

Agent

Chess-playing program

Robot controller

Game-playing AI

Driving system

Controller

Therefore:

Agent = Decision Maker

The Environment

The Environment is everything the agent interacts with.

After receiving A_t , the environment produces a reward and moves to a new state:

$$(S_t, A_t) \rightarrow (R_{t+1}, S_{t+1})$$

The environment's transition probability can be represented as:

$$P(S_{t+1} | S_t, A_t)$$

S_t = **current state** at time t

A_t = **action** taken at time t

S_{t+1} = **next state** after taking that action

$P(\cdot | \cdot)$ = **probability given/conditional on**

For example, in a robot navigation problem:

Agent = Robot

Environment =

├ Walls

├ Obstacles

├ Target

└ Physical space

The environment responds to the agent's actions.

The Environment

Simple example

Imagine a robot is in a room.

- S_t : Robot is **at the door**
- A_t : Robot chooses to **move right**
- S_{t+1} : Robot ends up **in the hallway**

Suppose there is a 90% chance it reaches the hallway:

$$P(S_{t+1} = \text{hallway} \mid S_t = \text{door}, A_t = \text{move right}) = 0.9$$

There might also be a 10% chance it stays at the door:

$$P(S_{t+1} = \text{door} \mid S_t = \text{door}, A_t = \text{move right}) = 0.1$$

This probability describes the **transition dynamics** of the environment: how the environment changes from one state to another after an action.

State

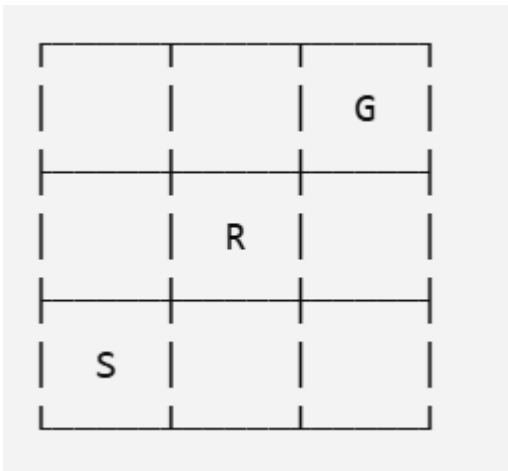
A **State** describes the environmental situation at a particular time.

We usually write:

$$S_t$$

Example: Robot

Suppose a robot moves on a grid:



If the robot is at row 2, column 2:

$$S_t = (2, 2)$$

A state can contain much more information than a position.

For example:

$$S_t = (\text{position}, \text{energy}, \text{enemy position})$$

Action

An **Action** is something the agent can do.

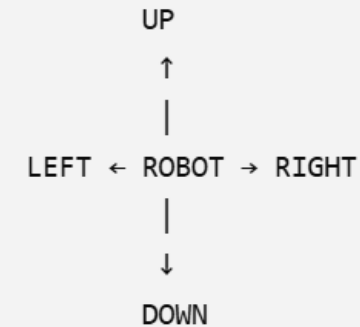
We write:

$$A_t$$

For a robot:

$$A = \{Up, Down, Left, Right\}$$

Figure



Therefore:

Action = What the agent can do

Reward

The **Reward** is feedback from the environment.

We normally write:

$$R_{t+1}$$

where R_{t+1} represents the **reward received after the agent takes action A_t** .

Example: Robot

Suppose a robot is moving on a grid and its goal is to reach a target.

- If the robot moves closer to the target, it may receive a small positive reward:

$$R_{t+1} = +1$$

- If the robot reaches the target, it may receive a large reward:

$$R_{t+1} = +10$$

- If the robot hits an obstacle, it may receive a negative reward:

$$R_{t+1} = -10$$

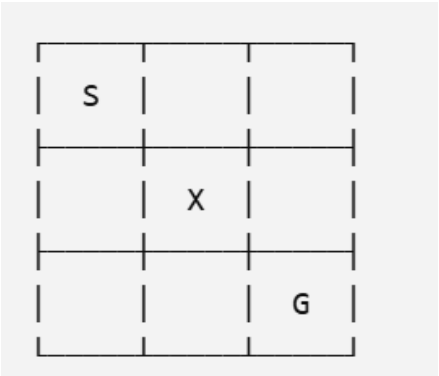
Therefore, the interaction can be represented as:

$$S_t \xrightarrow{A_t} S_{t+1}, \quad R_{t+1}$$

In simple terms, the reward tells the agent how desirable the result of its action was and helps the agent learn which actions to take in the future.

Complete Example: Grid World

Consider:



Where:

- S = Start
- G = Goal
- X = Obstacle

Suppose:

$$R = -1$$

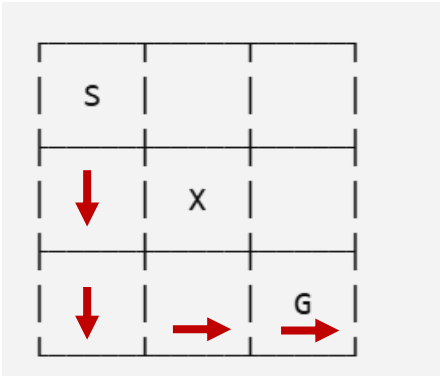
for every normal movement, and:

$$R = +10$$

When the agent reaches the goal.

The agent must learn a path such as:

$$S \rightarrow \text{Down} \rightarrow \text{Down} \rightarrow \text{Right} \rightarrow \text{Right} \rightarrow G$$



The goal is not simply to move.

The goal is to learn a **good sequence of decisions**.

Immediate Reward vs. Long-Term Reward

In Reinforcement Learning, an agent does not only consider the **reward it receives immediately**. It also considers the **future rewards** that it may receive. This is the difference between immediate reward and long-term reward.

1. Immediate Reward

An **immediate reward** is the reward that the agent receives directly after taking an action.

We write it as:

$$R_{t+1}$$

For example, suppose a robot is moving toward a target. If the robot moves one step closer to the target and receives a reward of $+1$, then:

$$R_{t+1} = +1$$

The immediate reward tells the agent how good or bad its **current action** was.

Immediate Reward vs. Long-Term Reward

2. Long-Term Reward

A **long-term reward** refers to the total reward that the agent expects to receive over multiple future steps. It is also called the **return**.

We write the return as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the **discount factor** and determines how much the agent values future rewards.

Return

The **Return** is the cumulative future reward.
Without discounting:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

Usually, however, we use a **discount factor**.
Therefore:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where:

$$0 \leq \gamma \leq 1$$

Discount Factor

The parameter γ

is called the **Discount Factor**.

It determines how much importance we give to future rewards.

If:

$$\gamma = 0$$

The agent cares only about immediate reward.

If:

$$\gamma \approx 1$$

The agent gives significant importance to future rewards.

Example: Robot

Suppose a robot has two choices:

- **Action A:** Move closer to the target and receive $+1$ immediately.
- **Action B:** Move away from the target and receive 0 immediately, but this allows the robot to reach the target later and receive $+10$.

The agent should consider not only:

$$\text{Immediate Reward} = R_{t+1}$$

but also:

$$\text{Long-Term Return} = G_t$$

For example, if the rewards are:

$$R_{t+1} = 0, \quad R_{t+2} = 0, \quad R_{t+3} = 10$$

then the long-term return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = 0 + \gamma(0) + \gamma^2(10)$$

$$G_t = 10\gamma^2$$

Therefore, **immediate reward** tells the agent what it receives now, while **long-term reward** considers the **rewards it can receive over time**. The goal of Reinforcement Learning is generally to learn actions that maximize the **long-term cumulative reward**, rather than simply choosing the action with the largest immediate reward.

Question?

Why is the long-term reward (return) defined by this equation?

Why do we use this particular formula instead of another one?

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Response

Imagine you have two choices:

- **Choice A:** receive \$100 now
- **Choice B:** receive \$100 after 10 years

If we simply add rewards:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots \quad G_t = 100 + 100$$

we treat both \$100 rewards as equally valuable.

But in many situations, receiving a reward **now** is more valuable than receiving the same reward much later.

So we use γ :

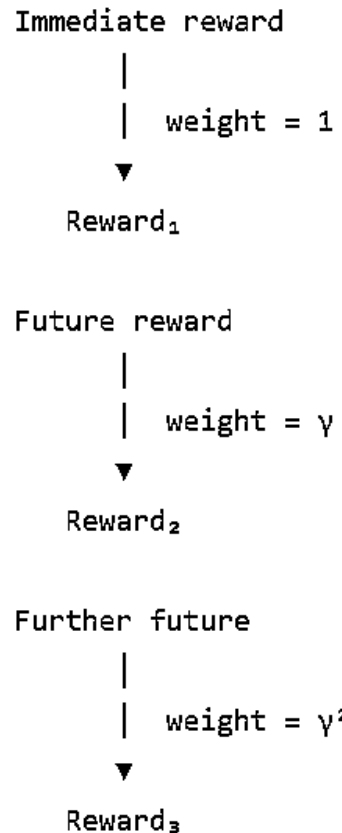
$$G_t = 100 + \gamma^{10}(100)$$

If $\gamma = 0.9$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad G_t = 100 + 0.9^{10}(100)$$

The future \$100 has a smaller contribution to today's decision.

Intuition



Example: Return

Suppose:

$$R_{t+1} = 10$$

and:

$$R_{t+2} = 5$$

$$R_{t+3} = 2$$

Calculate the return.

Solution

Therefore:

Discount Factor is $\gamma = 0.9$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = 10 + 0.9(5) + 0.9^2(2)$$

$$G_t = 10 + 4.5 + 1.62$$

$$G_t = 16.12$$

Policy

A **Policy** describes how an agent chooses actions.
We use:

 π

A stochastic policy is written as:

$$\pi(a|s)$$

This means:

The probability of selecting action a given state s .

Example

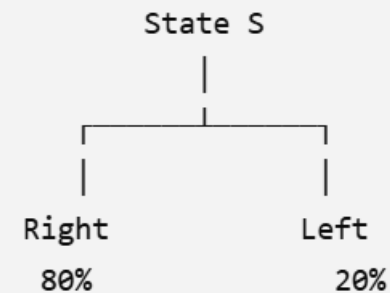
Suppose:

$$\pi(Right|S) = 0.8$$

and:

$$\pi(Left|S) = 0.2$$

Then:



Deterministic vs. Stochastic Policy

Deterministic Policy

One state always produces one action:

$$\pi(s) = a$$

Example:

$$\pi(S) = Right$$

Stochastic Policy

Actions have probabilities:

$$\pi(a|s)$$

Example:

$$\pi(Right|S) = 0.8$$

$$\pi(Left|S) = 0.2$$

Both types are important in RL.

Deterministic	One state → one fixed action
Stochastic	One state → different actions with probabilities
In short:	
	Deterministic: $s \rightarrow a$
	Stochastic: $s \rightarrow P(a \mid s)$

Exploration vs. Exploitation

One of the most important problems in RL is:

Exploration vs. Exploitation

Exploration

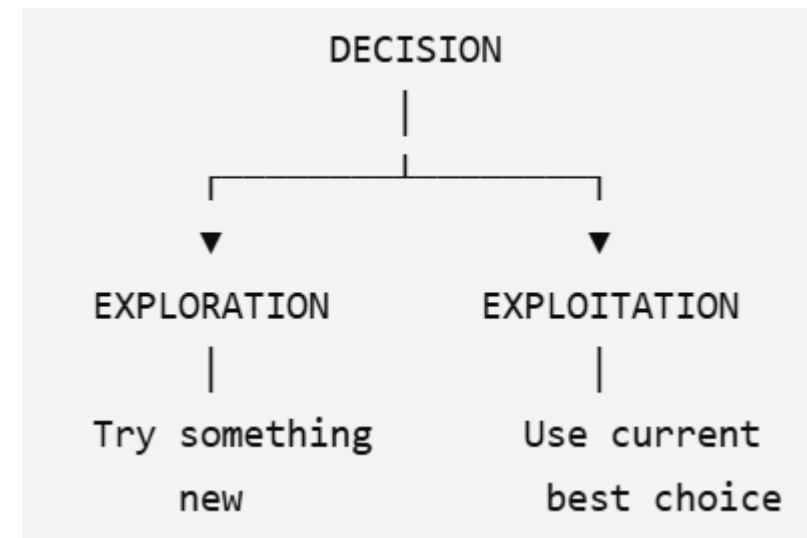
Try something new.

"Maybe there is a better action that I have not discovered yet."

Exploitation

Use the best action currently known.

"I already know that this action works well."



A good RL algorithm needs to balance both.

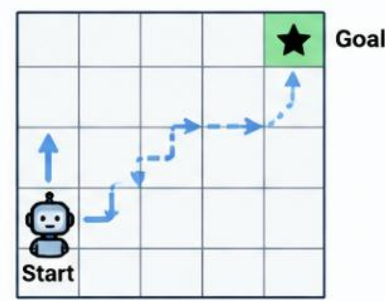
Exploration vs. Exploitation in Reinforcement Learning

In reinforcement learning, the agent must balance two goals:

Exploration (try new actions) and **Exploitation** (use what you already know works)

Exploration

Exploration means trying different actions to discover new states and find better rewards. It helps the agent learn about the environment.

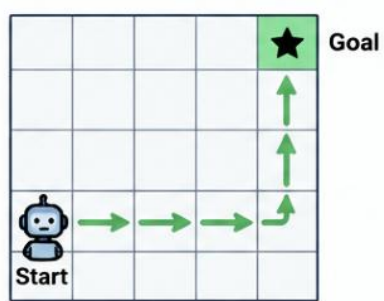


Example:
The agent tries moving up, right, left, etc., even if it is not the best action right now. This helps it find the goal and learn more about the environment.

Exploration = “Try and learn”

Exploitation

Exploitation means choosing the action that currently seems best based on what the agent has already learned. It helps the agent get higher rewards.



Example:
After learning that moving right and then up leads to the goal, the agent chooses this action repeatedly.

Exploitation = “Use what works”

The Trade-off

If the agent only explores, it may waste time trying random actions. If it only exploits, it may miss better rewards it hasn't found yet. So, the agent needs a good balance.

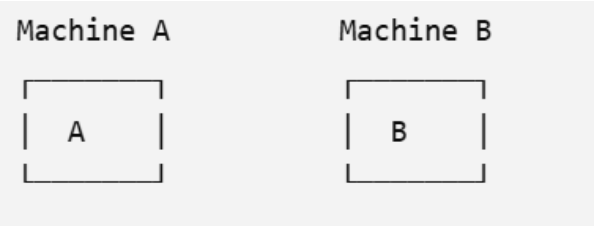


- Common ways to balance them:**
- ϵ -greedy (sometimes explore with probability ϵ)
 - Softmax (choose actions based on probabilities)
 - Upper Confidence Bound (UCB)
 - Other advanced methods (e.g., in deep RL)

In short: Exploration finds the treasure, Exploitation collects it!

Example: Multi-Armed Bandit Intuition problem

Imagine two machines:



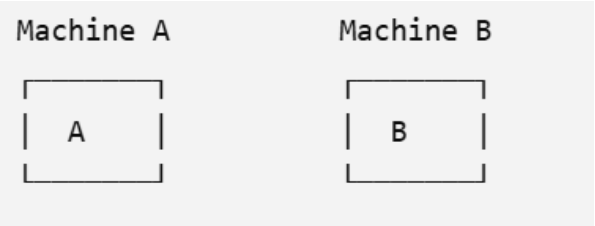
A and **B**. Each time an agent chooses a machine, it receives a reward.

The agent's goal is to choose machines that give it the **highest possible long-term reward**.

Round	Choice	Reward
1	A	1
2	B	0
3	A	1
4	B	1
5	A	0
6	B	1
7	A	1
8	B	1
9	A	0
10	B	1

Example: Multi-Armed Bandit Intuition problem

Imagine two machines:



Machine A

A was selected 5 times:

1, 1, 0, 1, 0

Total reward:

$$1 + 1 + 0 + 1 + 0 = 3$$

Average:

$$\hat{Q}(A) = \frac{3}{5} = 0.6$$

Machine B

B was selected 5 times:

0, 1, 1, 1, 1

Total reward:

$$0 + 1 + 1 + 1 + 1 = 4$$

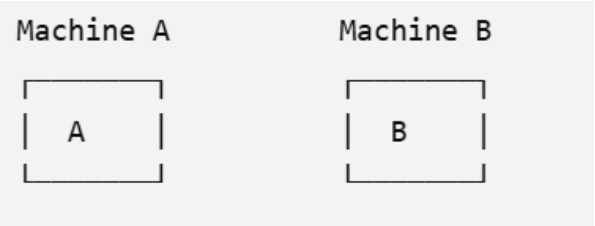
Average:

$$\hat{Q}(B) = \frac{4}{5} = 0.8$$

Now the agent has learned:

Example: Multi-Armed Bandit Intuition problem

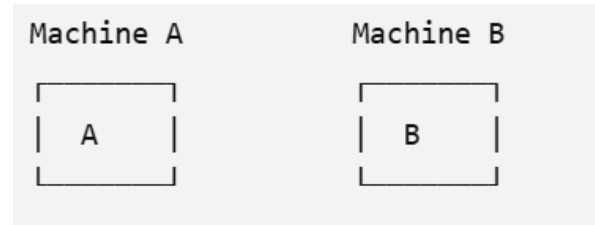
Imagine two machines:



Machine	Attempts	Total reward	Estimated reward
A	5	3	0.60
B	5	4	0.80

Example: Multi-Armed Bandit Intuition problem

Imagine two machines:



7. Exploitation

The agent can now use what it has learned.

Since:

$$\hat{Q}(B) = 0.8 > \hat{Q}(A) = 0.6$$

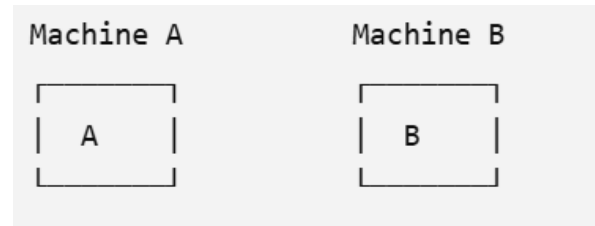
the agent will generally choose **B more often**.

This is **exploitation**:

"Based on what I've learned, B currently gives me more reward, so I'll use B."

Example: Multi-Armed Bandit Intuition problem

Imagine two machines:



8. But should it completely stop trying A?

Not necessarily.

This is where the problem becomes interesting.

Suppose the agent has tried:

- $A \rightarrow 5$ times
- $B \rightarrow 100$ times

It might have a very good estimate for B, but perhaps A hasn't been tested enough.

There is always some uncertainty.

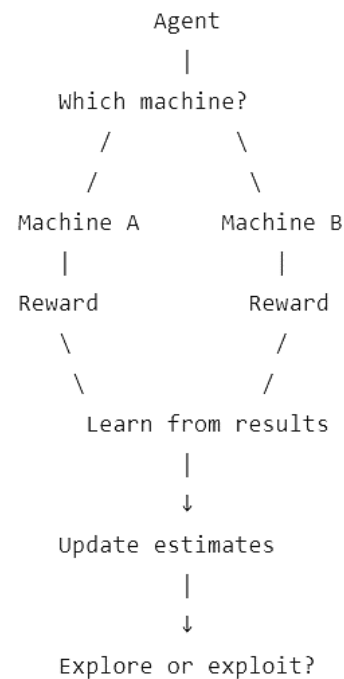
Therefore, a good algorithm might occasionally try A again.

That is **exploration**.

Example: Multi-Armed Bandit Intuition problem

9. Exploration vs. Exploitation

We can visualize the decision like this:



The agent repeatedly goes through this cycle:

Choose → Receive reward → Learn → Choose again

Example: Multi-Armed Bandit Intuition problem**10. Why is it called "Multi-Armed Bandit"?**

Imagine a casino with many slot machines.

Each slot machine is called an **arm**.

With two machines:

A, B

we have a **2-armed bandit**.

With 10 machines:

$A, B, C, D, \dots, J$

we have a **10-armed bandit**.

The agent must figure out:

Which arm should I pull to maximize my total reward?

11. The key mathematical idea

For each machine, the agent maintains an estimated value.

For machine A :

$$\hat{Q}(A) = \frac{\text{Total rewards from A}}{\text{Number of times A was chosen}}$$

Similarly:

$$\hat{Q}(B) = \frac{\text{Total rewards from B}}{\text{Number of times B was chosen}}$$

In our example:

$$\hat{Q}(A) = \frac{3}{5} = 0.6$$

and

$$\hat{Q}(B) = \frac{4}{5} = 0.8$$

Therefore, B currently has the higher estimated value.

12. The important lesson

The agent's goal is **not** simply:

"Choose the machine with the highest reward observed so far."

Instead, its goal is:

Learn which machine is likely to produce the highest long-term reward while continuing to collect rewards during the learning process.

That's why Multi-Armed Bandits are fundamentally about the **exploration–exploitation trade-off**.

In one sentence:

Explore to learn, exploit to earn.

And algorithms such as ϵ -greedy, UCB, and Thompson Sampling are different ways of deciding how much to explore versus how much to exploit.

Markov Decision Process

Now we introduce one of the most important mathematical frameworks in this course:

Markov Decision Process (MDP)

An MDP provides a mathematical framework for sequential decision-making under uncertainty.

A common representation is:

$$(S, A, P, R, \gamma)$$

where:	
Symbol	Meaning
S	Set of states
A	Set of actions
P	Transition probability
R	Reward function
γ	Discount factor

Transition Probability

Suppose the agent is in state s and chooses action a .

The environment may move to a new state s' .

We write:

$$P(s'|s, a)$$

This means:

The probability of reaching s' , given that the agent was in s and selected a .

Example

$$P(s'|s, a) = 0.8$$

means there is an 80% probability of reaching s' .

This allows MDPs to represent uncertainty.



The Markov Property

This is a very important concept.

The **Markov Property** says that the current state contains all the relevant information needed to predict the future.

Formally:

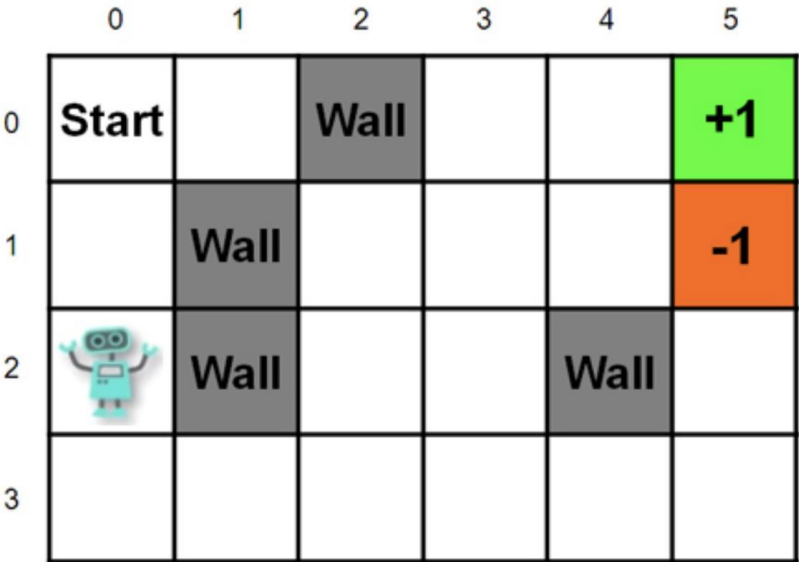
$$P(S_{t+1}|S_t, A_t, \text{history}) = P(S_{t+1}|S_t, A_t)$$

In simple words:

We do not need the entire history if the current state contains all relevant information.

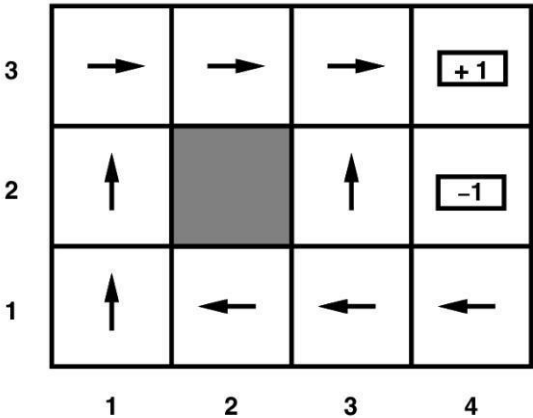
2. Simple Example: Robot

Imagine a robot moving around a small grid.



The robot wants to reach the **goal**.

Example 2: Grid world



Optimal policy when $R(s) = -0.04$ for every non-terminal state

3. State

A **state** describes the current situation of the agent.

For example:

$$S = (2, 2)$$

could mean:

The robot is currently at row 2, column 2.

Other possible states might be:

$$(1, 1), (1, 2), (1, 3), (2, 1), \dots$$

So:

State = Where/what the agent currently is

3. State

A **state** describes the current situation of the agent.

For example:

$$S = (2, 2)$$

could mean:

The robot is currently at row 2, column 2.

Other possible states might be:

$$(1, 1), (1, 2), (1, 3), (2, 1), \dots$$

So:

State = Where/what the agent currently is

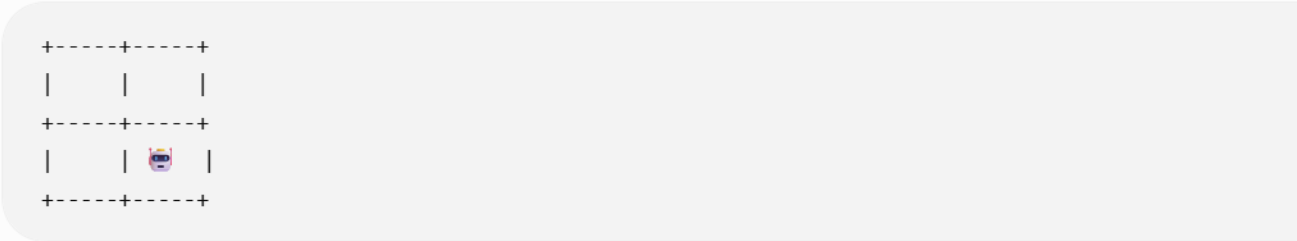
4. Action

From a state, the agent can choose an **action**.

For example:

$$A = \{ \text{Up, Down, Left, Right} \}$$

If the robot is here:



it could choose:

- ↑ Up
- ↓ Down
- ← Left
- → Right

So:

Action = What the agent decides to do

	0	1	2	3	4	5
0	Start		Wall			+1
1		Wall				-1
2		Wall			Wall	
3						

4. Action

From a state, the agent can choose an **action**.

For example:

$$A = \{\text{Up, Down, Left, Right}\}$$

If the robot is here:

```
+---+---+
|   |   |
+---+---+
|   | 🤖 |
+---+---+
```

it could choose:

- ↑ Up
- ↓ Down
- ← Left
- → Right

So:

Action = What the agent decides to do

5. Reward

After taking an action, the agent receives a **reward**.

For example:

- Reaching the goal $\rightarrow +10$
- Moving normally $\rightarrow -1$
- Falling into a trap $\rightarrow -10$

For example:

$$R = +10$$

means the agent received a positive reward.

The reward tells the agent whether the outcome was desirable.

6. Transition

After the agent takes an action, the environment changes.

Suppose the robot is at:

$$S_t = (2, 2)$$

and chooses:

$$A_t = \text{Right}$$

It might move to:

$$S_{t+1} = (2, 3)$$

This is called a **state transition**.

Mathematically:

$$S_t \xrightarrow{A_t} S_{t+1}$$

7. The Markov Property

The most important word in **Markov Decision Process** is **Markov**.

The Markov property says:

The future depends on the current state, not on the entire history of previous states and actions.

In other words:

$$P(S_{t+1} | S_t, A_t)$$

is enough to describe the next state.

We don't need:

$$S_1, A_1, S_2, A_2, \dots, S_t$$

if S_t already contains all the information needed.

Example of the Markov Property

Imagine a chess game.

If the current state contains:

- Position of every piece
- Whose turn is it
- Relevant game information

Then we do not need to remember every previous move to determine the legal next moves.

The current state summarizes the relevant history.

RL and MDP: What Is the Difference?

This is a very common exam question.

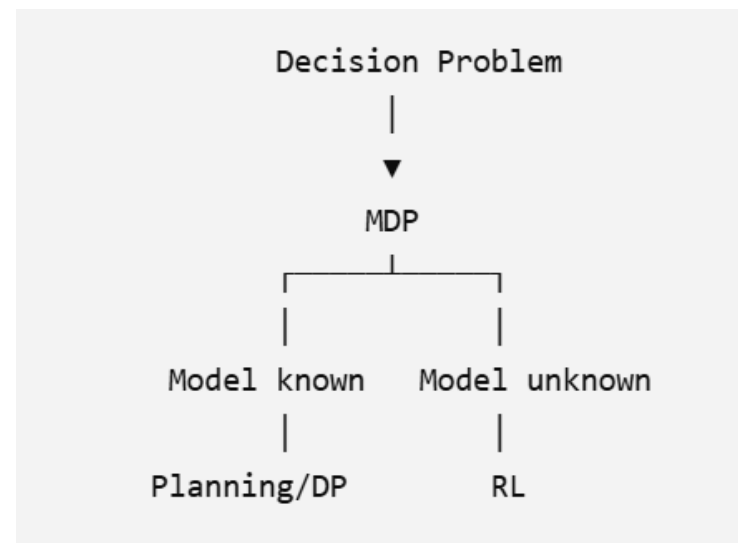
MDP

An MDP describes the **mathematical decision problem**.

Reinforcement Learning

RL describes how an agent can **learn a good decision-making strategy through interaction**.

A useful intuition is:



In many RL problems, the agent does not initially know the transition probabilities or reward structure.

It must learn from experience.

23. Full Example: Game Playing

Suppose an AI plays a game.

State

$$S_t$$

may contain:

- Player position
- Enemy positions
- Health
- Available resources
- Goal location

Actions

$$A_t$$

might be:

- Move
- Attack
- Defend
- Run

Rewards

Win game	+100
Defeat enemy	+20
Normal move	-1
Lose game	-100

Policy

$$\pi(a|s)$$

determines which action to choose.

Objective

The agent tries to maximize expected return:

$$\max_{\pi} E[G_t]$$

24. Solved Exercise 1

Question

An agent receives:

$$R_{t+1} = 5$$

$$R_{t+2} = 10$$

$$R_{t+3} = 20$$

with:

$$\gamma = 0.5$$

Calculate the return.

Solution

$$G_t = 5 + 0.5(10) + (0.5)^2(20)$$

$$G_t = 5 + 5 + 5$$

Therefore:

$$G_t = 15$$

25. Solved Exercise 2

Given:

$$\gamma = 0.9$$

and:

$$R_{t+1} = 10$$

$$R_{t+2} = 0$$

$$R_{t+3} = 10$$

Calculate G_t .

Solution

$$G_t = 10 + 0.9(0) + (0.9)^2(10)$$

$$G_t = 10 + 8.1$$

Therefore:

$$\boxed{G_t = 18.1}$$

26. Solved Exercise 3 — Policy

Suppose the agent is in state s_1 .

There are three possible actions:

$$A = \{Left, Right, Up\}$$

The policy is:

$$\pi(Left|s_1) = 0.2$$

$$\pi(Right|s_1) = 0.5$$

$$\pi(Up|s_1) = 0.3$$

Question

What is the probability of choosing Right?

Answer

$$\pi(Right|s_1) = 0.5$$

Therefore, the agent chooses Right with probability:

50%

Stochastic

One state → different actions with probabilities

27. Solved Exercise 4 — MDP

Suppose:

$$S = \{s_1, s_2\}$$

and:

$$A = \{Left, Right\}$$

If:

$$P(s_2|s_1, Right) = 0.8$$

what does this mean?

Answer

It means:

If the agent is currently in s_1 and chooses Right, there is an 80% probability that the next state will be s_2 .

28. Exam Question 1 — Definition

Question

Define Reinforcement Learning.

Model answer

Reinforcement Learning is a learning framework in which an agent learns to make decisions by interacting with an environment and receiving rewards, with the objective of maximizing long-term cumulative reward.

29. Exam Question 2 — Components

Question

What are the main components of an RL problem?

Answer

The main components include:

Agent, Environment, State, Action, Reward, Policy

The basic interaction is:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

30. Exam Question 3 — Return

Question

Define Return and give its mathematical formula.

Answer

Return is the cumulative discounted future reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the discount factor.

31. Exam Question 4 — Exploration vs. Exploitation

Question

Explain the difference between exploration and exploitation.

Answer

Exploration means trying actions whose consequences are not yet well known.

Exploitation means selecting actions that currently appear to provide the highest expected reward.

An RL agent needs to balance both.

32. Exam Question 5 — MDP

Question

Define a Markov Decision Process and list its components.

Answer

An MDP is a mathematical framework for sequential decision-making under uncertainty.

It is commonly represented as:

$$(S, A, P, R, \gamma)$$

where:

- S : states
- A : actions
- P : transition probabilities
- R : reward function
- γ : discount factor

33. Exam Question 6 — Markov Property

Question

What is the Markov Property?

Answer

The Markov Property states that the future depends on the current state and action rather than the complete history:

$$P(S_{t+1}|S_t, A_t, \text{history}) = P(S_{t+1}|S_t, A_t)$$

34. The Most Important Formulas

For your exam, these formulas are essential.

RL interaction

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

Return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Policy

$$\pi(a|s)$$

MDP

$$(S, A, P, R, \gamma)$$

Markov Property

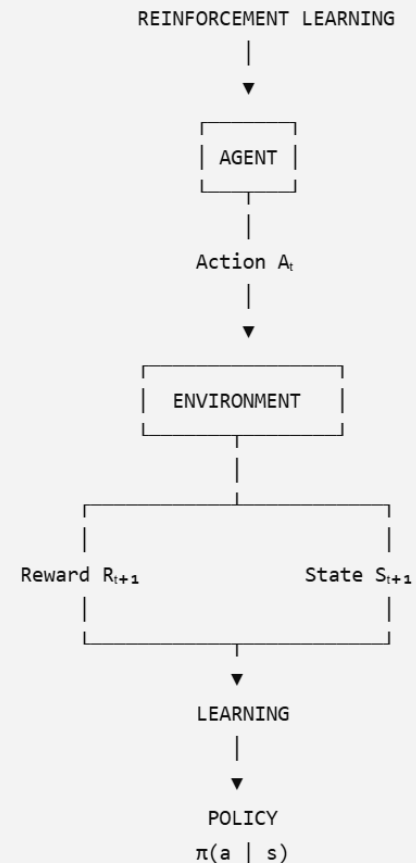
$$P(S_{t+1}|S_t, A_t, \text{history}) = P(S_{t+1}|S_t, A_t)$$

RL Objective

$$\max_{\pi} E_{\pi}[G_t]$$

Final Mental Model

If you remember only one diagram from Chapter 1, remember this



The entire subject can be understood initially as:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ New State $\rightarrow$ Learning

Then MDP gives us the mathematical framework for this process.

Before moving to Chapter 2, make sure you can answer these **without looking at the notes**:

1. What is Reinforcement Learning?
2. What is an Agent?
3. What is an Environment?
4. What is a State?
5. What is an Action?
6. What is a Reward?
7. What is a Policy?
8. What is Return?
9. What does γ mean?
10. What is Exploration?
11. What is Exploitation?
12. What is an MDP?
13. What does $P(s'|s, a)$ mean?
14. What is the Markov Property?
15. What is the difference between RL and supervised learning?

Thank you for your attention.

Any questions?

Assignments and Course Preparation

A. Chapter Preparation

Please read the chapter and complete the mini-project and exercises. Submit them by this Wednesday.

B. Review of the Previous Session

As an assignment and to review the concepts from the previous session, I will ask each student the questions.

C. Course Materials

The slides for this chapter, the lecture notes, and the exercises on the specified pages will be uploaded to the course page.

D. Teaching Assistant (TA)

Please send me the name of one person who could be selected as the TA.

E. Student Presentations

Each student should choose a topic related to *Reinforcement Learning and Decision Processes* and specify their preferred presentation date.

Thank you!